

FreeRTOS Advanced I

Manajemen Memori, Sinkronisasi Kompleks, dan Eksekusi Kritis pada Sistem Real-Time

Abby Putro Nugroho

40040324620017

TRO - 2024 - A

Task Notifications

Alternatif Super Ringan

Fitur asli setiap Task Control Block (TCB) berupa array notifikasi 32-bit. Dirancang sebagai pengganti Queue/Semaphore jika data ditujukan langsung ke satu task spesifik.

Keunggulan: Mengeksekusi instruksi CPU ~45% lebih cepat dan **tidak** membutuhkan alokasi memori dinamis (RAM/Heap) tambahan karena menempel pada TCB.

Penggunaan Multiguna

- **Event Signaling:** Mensimulasikan Binary Semaphore.
- **Pipeline Data Ringan:** Mampu mengirim sebuah angka integer 32-bit (Set Value) sebagai payload.
- **Counting Semaphore:** Notifikasi bisa di-increment dan di-decrement berulang kali layaknya counter.

Event Groups (Bit Flags)



Sinkronisasi Majemuk

Memungkinkan sebuah task memantau hingga 24 kejadian (event flags) berbeda secara simultan hanya dalam satu variabel berukuran word (32-bit).



Logika AND / OR

AND (Barrier): Task menunggu sampai Sensor Suhu Siap DAN Sensor Tekanan Siap. **OR:** Task terbangun jika Tombol Ditekan ATAU Waktu Habis.



Distributed State

Sangat ideal untuk sistem Multi-MCU. Node pusat mengumpulkan bit status (flags) dari node-node periferal sebelum memulai proses inisialisasi utama (master control).

Software Timers

Arsitektur Timer Kernel

- **Non-Blocking:** Tidak menggunakan hardware timer, melainkan dikelola oleh sistem *System Tick* milik RTOS.
- **One-Shot vs Periodik:** Berjalan sekali (seperti watchdog timeout tunggal) atau terus-menerus (auto-reload).
- **Task Daemon:** Fungsi *Callback* timer tidak berjalan di task milik pengguna, melainkan dieksekusi di *Daemon Task* bawaan (RTOS Timer Service).
- **Aturan Emas:** DILARANG memanggil fungsi blocking (seperti `vTaskDelay`) di dalam callback timer, karena akan mengunci sistem seluruh timer.



Oscilloscope showing periodic square wave signals

Varian Semaphore & Mutex

Tipe	Deskripsi & Mekanisme Internal
Binary Semaphore	Bertindak sebagai sinyal/flag (1 atau 0). Sempurna untuk memberi tanda dari ISR (Hardware Interrupt) ke Task pemroses bahwa sebuah kejadian telah selesai.
Counting Semaphore	Memiliki nilai maksimal > 1 . Digunakan untuk manajemen sumber daya (resource). Contoh: antrian pool memori di mana setiap task menurunkan nilai counter saat mengambil blok memori.
Mutex + Inheritance	Hanya boleh dilepas (Give) oleh task yang menguncinya (Take). Dilengkapi Priority Inheritance : secara otomatis menaikkan prioritas task pemegang kunci jika task prioritas tinggi sedang mengantri untuk mencegah <i>Priority Inversion</i> .

Dynamic Memory Allocation (Heap)

Skema Heap 1-5 FreeRTOS

- **heap_1:** Alokasi memori yang tidak dapat dibebaskan. Paling aman dari error komputasi.
- **heap_2:** Memungkinkan pembebasan (free), tapi sangat rentan terhadap fragmentasi parah.
- **heap_4:** Standar industri. Memiliki algoritma *Coalescence* untuk menggabungkan blok-blok memori kosong yang bersebelahan.
- **heap_5:** Sama seperti heap_4, namun bisa menyatukan memori fisik yang terpisah lokasinya (contoh: Internal SRAM + External SPI PSRAM).

Ancaman Fragmentasi

Memori yang berlubang-lubang (pecah) menyebabkan sistem menolak pembuatan Task/Queue baru yang butuh array blok besar meskipun total RAM sisa masih cukup banyak.

Static Allocation (Safety-Critical)



Zero Dynamic Heap

Setiap Task, Queue, atau Mutex tidak dialokasikan menggunakan `malloc()` saat runtime. Sebagai gantinya, memori Stack dan TCB disediakan sebagai array global (RAM Statis) saat kompilasi kode (Compile Time).



Medical & Aerospace

Menggunakan API berekstensi khusus, contoh: `xTaskCreateStatic()`. Metode ini menjamin aplikasi Anda tidak mungkin gagal berjalan (crash) akibat *Out of Memory Exception* di tengah operasi.

Critical Section vs Mutex

Perbedaan Fundamental

- **Mutex (OS Level):** OS memblokir task lain dari menggunakan *resource* yang sama. Interupsi perangkat keras tetap berjalan normal.
- **Critical Section (Hardware Level):** Memanggil `taskENTER_CRITICAL()`. Mematikan (disable) seluruh interupsi global.
- **Peruntukan:** Critical Section ditujukan untuk operasi atomis yang **sangat singkat** (misal manipulasi bit register tunggal), karena menahan interrupt akan merusak waktu nyata sistem (real-time).



Magnifying glass focusing on a microchip

Task Suspension

vTaskSuspend()

Memaksa sebuah task keluar dari penjadwalan Scheduler tanpa batas waktu dan tanpa menghapusnya dari RAM. Sangat efisien untuk task yang fungsinya hanya berjalan musiman (misalnya task setup/pairing WiFi).

vTaskResume()

Fungsi pemicu untuk membangkitkan kembali task yang ditidurkan. Memastikan struktur kode dan stack memori historis task tersebut kembali tereksekusi tepat di baris instruksi terakhirnya.

Stream Buffers & Message Buffers

Optimasi Pipa Data

 Binary data stream representation

- Evolusi dari Queue standar yang dirancang spesifik untuk satu penulis dan satu pembaca (Single Writer to Single Reader).
- **Stream Buffers:** Mengalirkan aliran *byte* kontinu tanpa peduli batas frame. Menggunakan **Trigger Level**: Task penerima baru terbangun jika buffer telah terisi sejumlah *X* byte (sangat efisien untuk UART DMA RX).
- **Message Buffers:** Varian cerdas. Setiap paket data berukuran dinamis yang dimasukkan akan secara otomatis disisipi *Header Panjang Data* tersembunyi. Saat dibaca, frame selalu keluar secara utuh.

Queue Sets



Konsep Master Listener

Mekanisme yang memungkinkan satu Task untuk diam/memblokir dan memantau (listen) ke banyak objek (Queue maupun Semaphore) sekaligus dari berbagai sumber.



Cara Kerja Sistem

Gunakan `xQueueSelectFromSet()`. Saat ada data masuk dari objek manapun (entah itu data Sensor dari Queue atau Semaphore Tombol dari ISR), task utama akan terbangun dan memilah objek mana yang aktif duluan untuk diproses.

Koordinasi Multi-MCU (Terdistribusi)

Protokol RTOS Jarak Jauh

- Mengabungkan ekosistem dua MCU (Misal STM32 untuk sensor keras, ESP32 untuk komputasi Awan).
- **Serial Bridge:** Memanfaatkan UART dengan DMA tinggi untuk memindahkan *State* RTOS.
- **Terdistribusi:** Semaphore yang memicu (Give) dari STM32, diterjemahkan menjadi paket protokol yang dipecah oleh ESP32 untuk melepaskan Event Group di jaringan internal ESP32 secara instan.



Two circuit boards connected with wires representing communication

Questions?

Terima kasih atas perhatian Anda dalam modul FreeRTOS Advanced I.

Abby Putro Nugroho

40040324620017

TRO - 2024 - A